Chapter 4

Functions, closures and generators

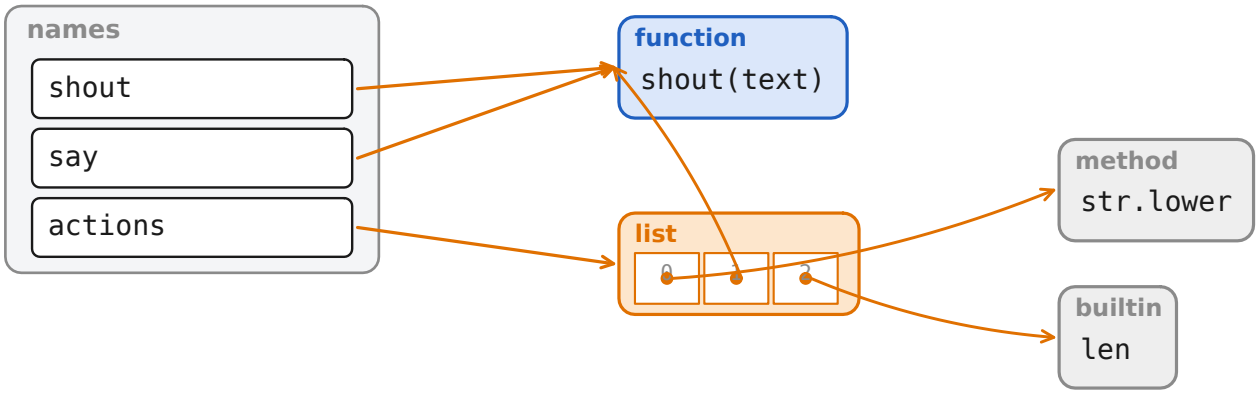
a function is an object you can pass, store, wrap and pause

You know how to define and call functions. This chapter is about what a function **is**: an object, created when `def` runs, bound to a name like any other value. Everything else follows: passing functions to functions, functions that remember things, functions that wrap other functions, and functions that pause halfway.

```
def shout(text):
    return text.upper() + "!"

say = shout                       # another name, same object
actions = [str.lower, shout, len]
print(say("hi"), [act("Hey") for act in actions])
print(type(shout).__name__, shout.__name__, shout.__code__.co_varnames)
```

```
HI! ['hey', 'HEY!', 3]
function shout ('text',)
```

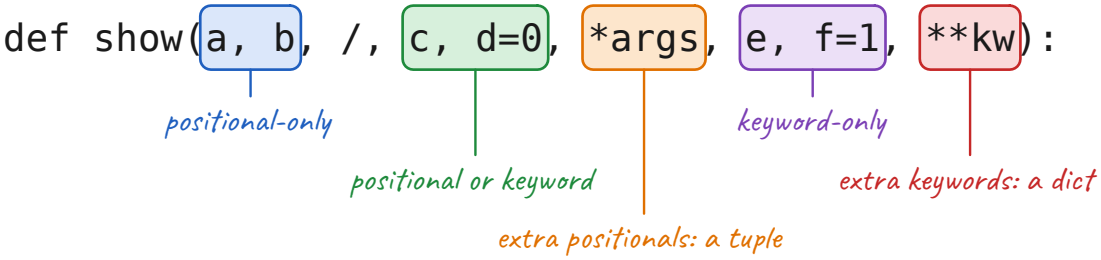


KEY IDEA

`def` is an assignment statement: it builds a function object (code, defaults, name, docstring) and binds it to a name. Calling is a separate step, written with `()`.

Parameter kinds: one signature, five zones

A signature can have up to five zones, always in this order. The `/` and `*` markers are walls: parameters before `/` can't be passed by name; parameters after `*` (or `*args`) must be:




```python
def show(a, b, /, c, d=0, *args, e, f=1, **kw):
    print(a, b, c, d, args, e, f, kw)

show(1, 2, 3, e=5)
show(1, 2, c=3, d=4, e=5, colour="red")
show(1, 2, 3, 4, 98, 99, e=5, f=6)
```

```
1 2 3 0 () 5 1 {}
1 2 3 4 () 5 1 {'colour': 'red'}
1 2 3 4 (98, 99) 5 6 {}
```

```python
show(1, b=2, c=3, e=5)
```

```
TypeError: show() missing 1 required positional argument: 'b'
```

you want...	write	why
a flag that must be named	`def save(path, *, overwrite=False)`	`save(p, True)` is unreadable; this forces `overwrite=True`
any number of values	`def total(*nums)`	`nums` is a tuple
to forward everything	`def wrapper(*args, **kwargs)`	decorators (later this chapter)
to spread a list into a call	`f(*items)`, `f(**options)`	the call-side mirror of `*args`
names you may rename later	`def area(w, h, /)`	callers can't depend on the names

The default-value trap

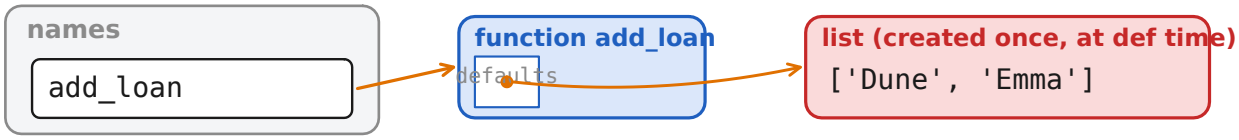
Defaults are evaluated **once**, when `def` runs, and stored on the function object. A mutable default is therefore shared by every call that doesn't pass that argument:

```python
def add_loan(title, loans=[]):
    loans.append(title)
    return loans

print(add_loan("Dune"))
print(add_loan("Emma"))           # surprise
print(add_loan.__defaults__)      # the one shared list
```

← *Dune is still there*

```
['Dune']
['Dune', 'Emma']
(['Dune', 'Emma'],)
```



The fix is the `None` sentinel: make the default immutable, create the list inside the call.

```python
def add_loan(title, loans=None):
    if loans is None:
        loans = []                # a new list on every call
    loans.append(title)
    return loans

print(add_loan("Dune"), add_loan("Emma"))
```

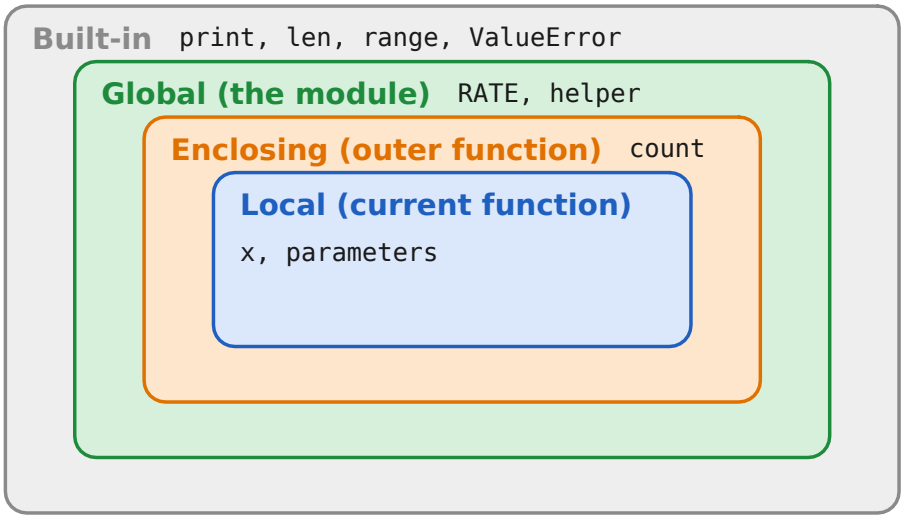
```
['Dune'] ['Emma']
```

WHY

This isn't a bug in Python; it falls out of "`def` runs once". The same rule makes `def stamp(t=time.time())` record the time the module loaded, not the time of the call.

Scope: where Python looks a name up

When code uses a name, Python searches four scopes in order, **L**ocal, **E**nclosing, **G**lobal, **B**uilt-in, and takes the first match:



Which scope a name belongs to is decided at **compile** time: if a function assigns to a name anywhere in its body, that name is local everywhere in that body. That produces this classic error:

```
count = 0

def bump():
    count += 1        # assignment makes count local, so it's read before being set

bump()
```

```
UnboundLocalError: cannot access local variable 'count' where it is not associated wi…
```

```
def make_counter():
    count = 0
    def bump():
        nonlocal count      # rebind the enclosing function's name
        count += 1
        return count
    return bump

tick = make_counter()
print(tick(), tick(), tick())
```

← *global count would mean the module's count*

```
1 2 3
```

RULE OF THUMB

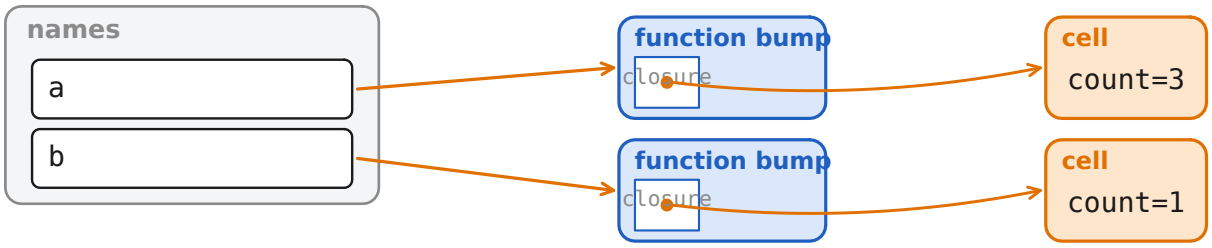
Read globals freely, but avoid `global` for writing; pass values in and return them out. Module-level constants (`RATE = 0.2`) are fine; module-level mutable state is a source of bugs.

Closures: functions that remember

`tick` above kept its own `count` after `make_counter` had returned. The inner function holds a reference to a **cell** that holds the variable. Each call of `make_counter` makes a new cell, so counters are independent:

```
a, b = make_counter(), make_counter()
a(); a()
print(a(), b())
print(a.__closure__[0].cell_contents, a.__code__.co_freevars)
```

```
3 1
3 ('count',)
```



The late-binding trap

A closure captures the **variable**, not its value at the time. Functions made in a loop all see the loop variable's **final** value:

captures the variable (bug)

```
fs = [lambda: i for i in range(3)]
print([g() for g in fs])
```

```
[2, 2, 2]
```

binds the value now (fix)

```
fs = [lambda i=i: i for i in range(3)]
print([g() for g in fs])
```

```
[0, 1, 2]
```

> **IN OTHER LANGUAGES**
> JavaScript closures work the same way (and had the same loop bug with `var`, fixed by `let`). Java lambdas may only capture variables that never change, which rules the bug out.

Functions that take functions

Because functions are values, a lot of the standard library takes a function as an argument ("higher-order" functions). The most useful by far is `key=`: tell `sorted`/`min`/`max` **what to compare**, not how to sort:

```python
from operator import itemgetter, attrgetter
from functools import partial

books = [("Dune", 1965, 4.3), ("Emma", 1815, 4.0), ("Ubik", 1969, 4.1)]
print(sorted(books, key=itemgetter(1))[0][0])          # oldest
print(max(books, key=lambda b: b[2])[0])               # best rated
print(sorted(books, key=lambda b: (-b[2], b[0]))[0])   # rating desc, then title

to_int = partial(int, base=2)                          # fix some arguments
print(to_int("1011"))
```

```
Emma
Dune
('Dune', 1965, 4.3)
11
```

map and filter

```python
nums = range(10)
out = list(map(lambda n: n * n,
               filter(lambda n: n % 2, nums)))
print(out)
```

```
[1, 9, 25, 49, 81]
```

a comprehension

```python
nums = range(10)
out = [n * n for n in nums if n % 2]

print(out)
```

```
[1, 9, 25, 49, 81]
```

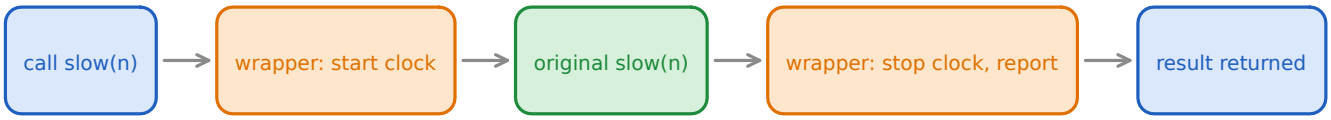


RULE OF THUMB

Prefer a comprehension when you'd need a `lambda`; `map(str, xs)` with an existing function is fine. `itemgetter`/`attrgetter` beat `lambda x: x[1]` for speed and clarity.

Decorators: wrapping a function

A decorator is a function that takes a function and returns a replacement, usually a wrapper that does something before and after calling the original. `@timed` above a `def` is just shorthand for `slow = timed(slow)`:




```
1  import functools, time
2
3  def timed(func):
4      @functools.wraps(func)                # copy name, docstring etc. onto the wrapper
5      def wrapper(*args, **kwargs):
6          start = time.perf_counter()
7          result = func(*args, **kwargs)
8          ms = (time.perf_counter() - start) * 1000
9          print(f"{func.__name__} took {ms:.3f} ms")
10         return result
11     return wrapper
12
13 @timed
14 def slow(n):
15     return sum(range(n))
16
17 print(slow(10_000), slow.__name__)
```

```
slow took 0.085 ms
49995000 slow
```

line 4: without it: __name__ == 'wrapper'

line 5: forwards any arguments

A decorator that takes options is one more layer: a function that **returns** a decorator.

```python
def retry(times):
    def decorate(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except ConnectionError:
                    if attempt == times:
                        raise
        return wrapper
    return decorate

calls = []
@retry(times=3)
def flaky():
    calls.append(1)
    if len(calls) < 3:
        raise ConnectionError("try again")
    return "ok"

print(flaky(), len(calls))
```

```
ok 3
```

The standard library's decorators

```python
import functools, time

@functools.cache                    # remember every result (lru_cache(maxsize=None))
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

t = time.perf_counter(); fib(300); ms = (time.perf_counter() - t) * 1000
print(fib(300) % 1000, ms < 50, fib.cache_info().hits)
```

```
600 True 299
```

decorator	does
`@functools.cache`, `@functools.lru_cache(maxsize=128)`	memoise results (arguments must be hashable)
`@property`, `@staticmethod`, `@classmethod`	change how a method is accessed (Ch 5)
`@dataclasses.dataclass`	write `__init__`, `__repr__`, `__eq__` for you (Ch 5)
`@contextlib.contextmanager`	turn a generator into a `with` block (Ch 6)
`@functools.singledispatch`	choose an implementation by argument type

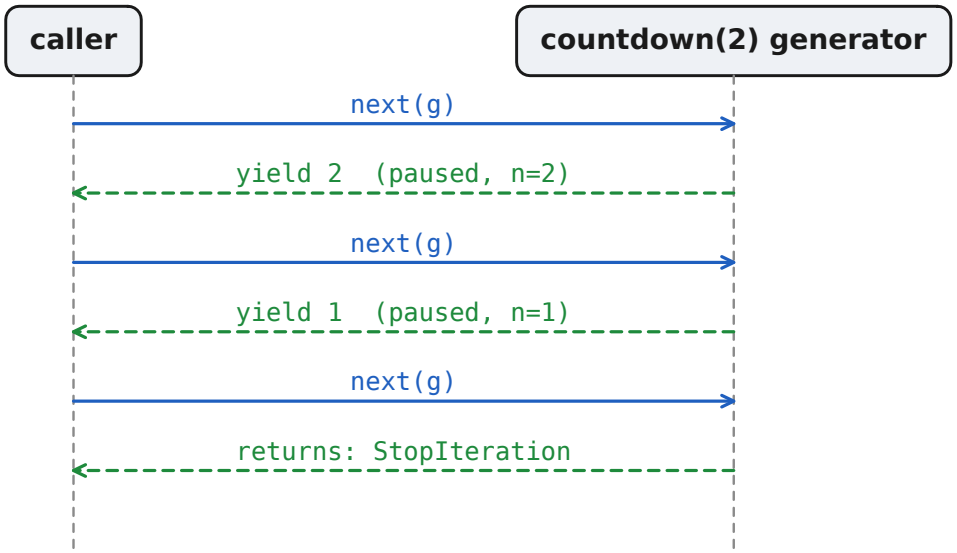
Generators: functions that pause

A function containing `yield` doesn't run when called. It returns a **generator**; each `next()` runs the body up to the next `yield`, hands out that value, and freezes there, with all its local variables kept, until the next `next()`:

```python
def countdown(n):
    print("  (starting)")
    while n > 0:
        yield n
        n -= 1
    print("  (done)")

g = countdown(2)
print("made it:", type(g).__name__)
print(next(g))
print(next(g))
print(list(g))           # runs to the end: StopIteration ends the list
```

```
made it: generator
  (starting)
2
1
  (done)
[]
```



Generators are lazy: they compute one item at a time, so a pipeline of them works on data far bigger than memory. Each stage pulls from the one before:

```python
import sys

def read_lines(n):                    # pretend this is a 10 GB log file
    for i in range(n):
        yield f"2026-09-{i % 30 + 1:02} GET /book/{i % 7}"

lines = read_lines(1_000_000)
hits = (ln for ln in lines if ln.endswith("/book/3"))     # generator expression
days = (ln[:10] for ln in hits)
print(next(days), sum(1 for _ in days) + 1)
print(sys.getsizeof(days), "bytes, vs",
      sys.getsizeof([0] * 1_000_000), "for a list of a million")
```

```
2026-09-04 142857
208 bytes, vs 8000056 for a list of a million
```

TRAP

A generator can only be consumed **once**. A second `for` over the same generator gets nothing, silently. If you need two passes, make a list, or call the generator function again.

Type hints: documentation a tool can check

Annotations say what a function expects and returns. Python itself **ignores** them at run time; they're read by humans, editors and checkers like mypy (Ch 17):

```python
from collections.abc import Iterable

def average(marks: Iterable[float], *, ndigits: int = 1) -> float | None:
    marks = list(marks)
    return round(sum(marks) / len(marks), ndigits) if marks else None

print(average([60, 71.5]), average([]), average.__annotations__["return"])
print(average("12"))     # the hint says numbers; Python only notices when sum() fails
```

```
65.8 None float | None
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

HONEST NOTE

Type hints don't make Python statically typed. They catch a class of bugs early **if** you run a checker. For a passport answer: Python is dynamically typed, with optional, gradual static typing on top. TypeScript did the same for JavaScript.

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 4.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 4.1` shows other valid solutions and their trade-offs.

`4.1` fix the bug Two bugs that every Python programmer writes once
`4.2` build from a spec Your own memoising decorator
`4.3` build from a spec Check types at run time from the hints
`4.4` build from a spec Lazy chunks and windows
`4.5` refactor, tests stay green Name your flags
`4.6` mini-project Mini-project: an event bus with decorators

PASSPORT STAMP ask the same question of every new language (Appendix B)

5. Functions and scope

Functions are first-class objects made at `def` time. Parameters: positional-only / normal / `*args` / keyword-only / `**kwargs`. Defaults evaluated once. LEGB scope fixed at compile time; closures capture variables (cells); decorators wrap.

4. Control flow

Generators: `yield` pauses a function; iteration pulls values lazily until `StopIteration`.

3. Typing

Dynamic typing with optional annotations, checked only by external tools.

DEEP DIVE (your vault)

Vault notes: 8.2 Parameters and arguments · 8.5 Defaults are evaluated at definition time · 8.7 Namespaces and the LEGB rule · 8.8 First-class functions · 8.9 Closures · 8.13 Decorators derived from first-class functions · 8.14 Decorator factories and timing · 6.9 Generator functions and suspension · 6.10 Generator expressions · 11.13 functools · 12.18 Type hints.